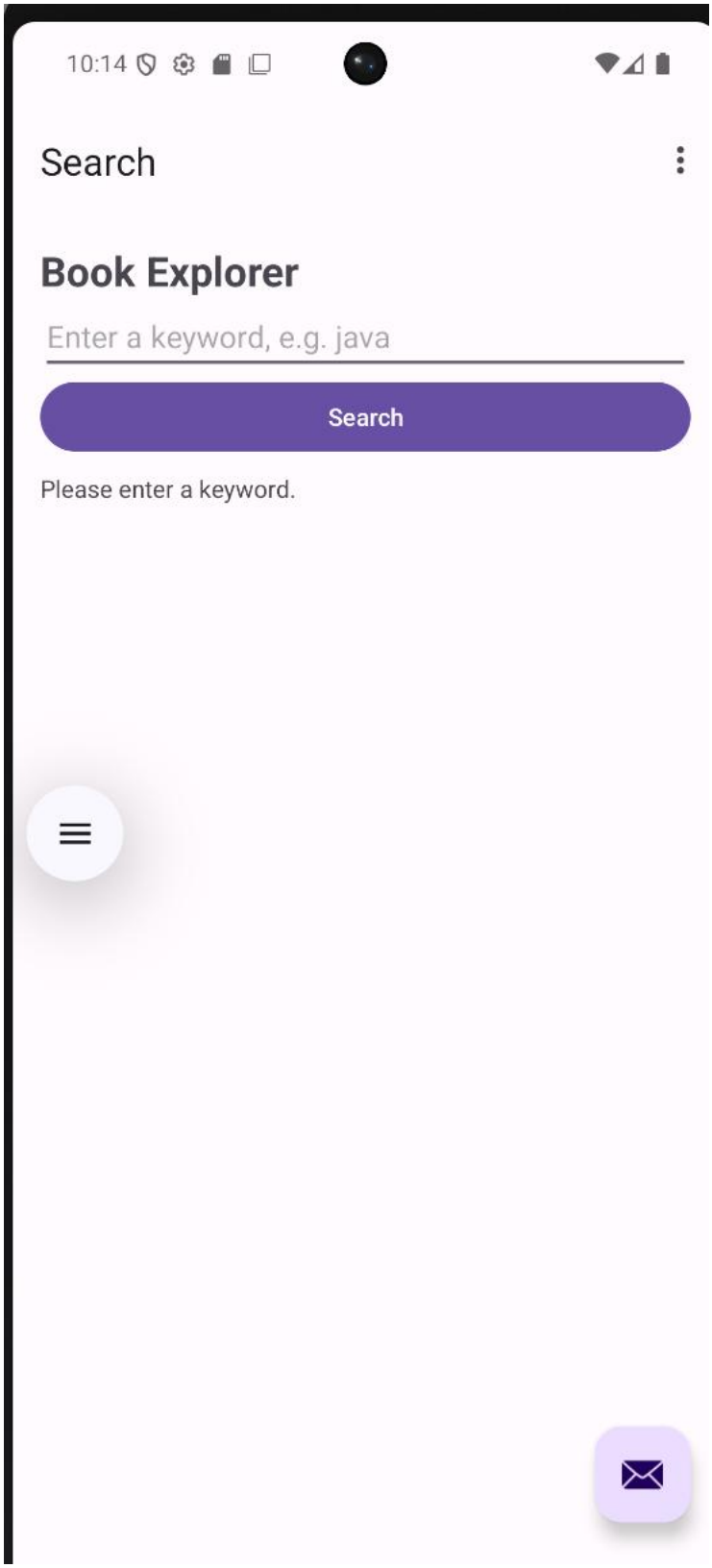
Project 4 Task 2 Writeup

1. Application Overview

My application is a Book Explorer app. The Android application allows a user to enter a keyword and search for books. The Android client sends the request to my cloud-hosted web service, not directly to the third-party API. The web service queries the Open Library API, processes the response, and returns simplified JSON data to the Android app. The web service also logs request information to MongoDB Atlas and provides a web-based dashboard that displays usage analytics and formatted request logs. This overall design matches the project requirement of a distributed application consisting of a mobile app, a cloud web service, and a dashboard.



2. Requirement 1: Native Android Application

The Android portion of my project satisfies the native application requirements.

2.1 Different kinds of Views

My Android layout uses at least four different kinds of Views:

EditText for keyword input
Button for submitting the search
TextView for status messages and book details
ListView for displaying search results

This satisfies the requirement that the application include at least three different kinds of Views.

2.2 Requires input from the user

The user enters a keyword such as java into the EditText field.

2.3 Makes an HTTP request to the web service

When the user taps the Search button, the Android app sends an HTTP GET request to my cloud web service at the /searchBooks endpoint.

2.4 Receives and parses JSON

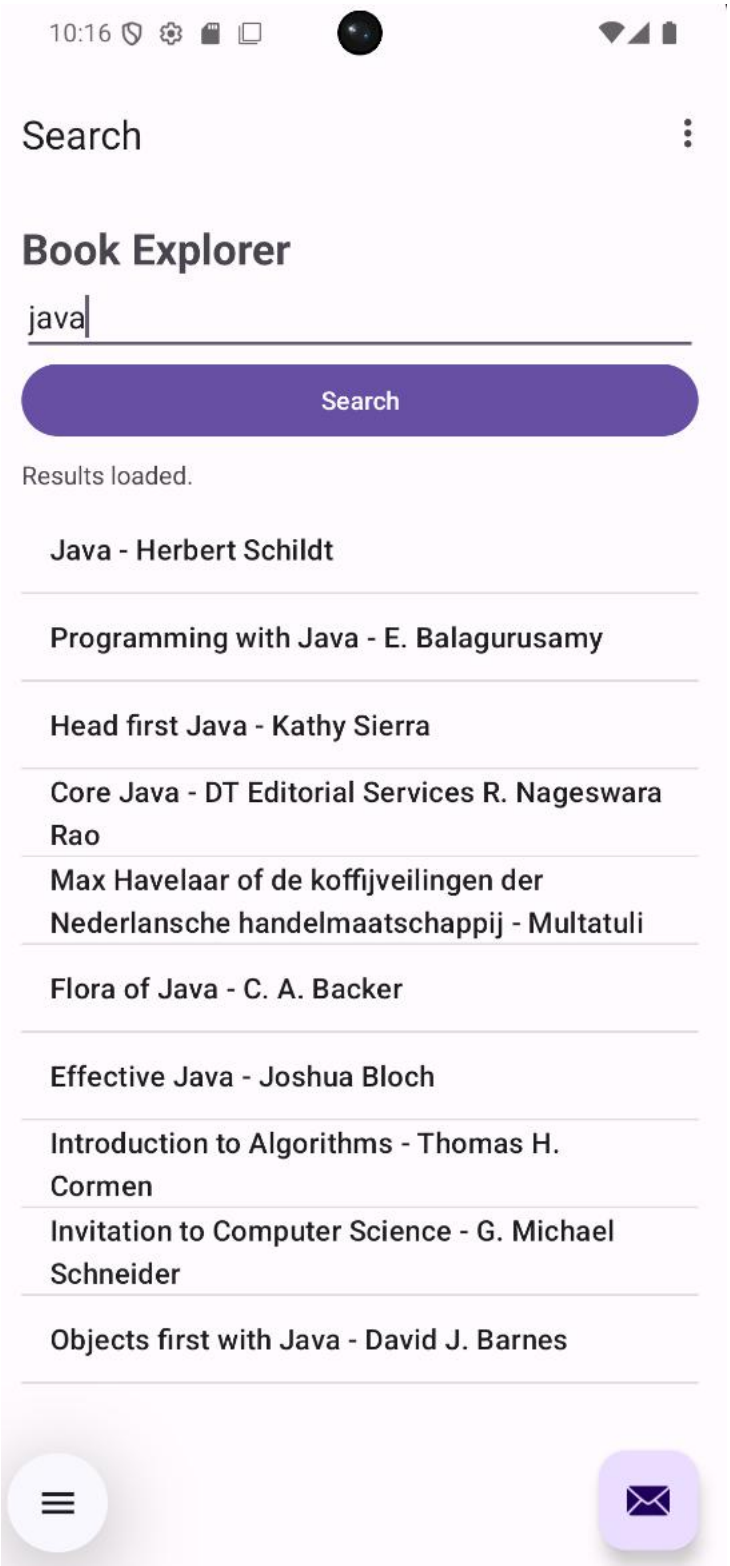
The Android app receives a JSON response from the web service. It parses the books array and extracts fields such as title, author, first publication year, and ISBN.

2.5 Displays new information to the user

The search results are displayed in a ListView. When the user taps one result, the app navigates to a detail screen that displays more information about the selected book.

2.6 Repeatable use

The application is repeatable because the user can enter different keywords and perform multiple searches without restarting the app.



Search results displayed in the Android app after entering a keyword.

Information for one selected book.

3. Requirement 2: Web Service

My web service is implemented using Servlets, as required by the project. The service is deployed to the cloud and provides a simple REST-style API for the Android application. The project instructions explicitly require Servlets rather than JAX-RS.

My main API endpoint is:

/searchBooks?keyword=<user_keyword>

The servlet receives a request from the Android app and extracts the keyword parameter.

The servlet performs the following business logic:

Validates the keyword input
Sends a request to the Open Library API
Parses the Open Library response
Extracts only the fields needed by the mobile application
Returns a simplified JSON response to the Android client

This satisfies the requirement that the web service receive an HTTP request from the native Android app, execute business logic, and fetch/process XML or JSON information from a third-party API.

3.4 Simplified JSON response
Example response format:


```
{
  "resultCount": 10,
  "books": [
    {
      "author": "Herbert Schildt",
      "isbn": null,
      "firstPublishYear": 1997,
      "openLibraryKey": "/works/OL720123W",
      "title": "Java"
    },
    {
      "author": "E. Balagurusamy",
      "isbn": null,
      "firstPublishYear": 2001,
      "openLibraryKey": "/works/OL22705232W",
      "title": "Programming with Java"
    }
  ]
}
```

ubiquitous-space-yodel-r97r65947gf5q6q-8080.app.github.dev/searchBooks?keyword=java

Pretty-print ☑


```
{
  "resultCount": 10,
  "books": [
    {
      "author": "Herbert Schildt",
      "isbn": null,
      "firstPublishYear": 1997,
      "openLibraryKey": "/works/OL720123W",
      "title": "Java"
    },
    {
      "author": "E. Balagurusamy",
      "isbn": null,
      "firstPublishYear": 2001,
      "openLibraryKey": "/works/OL22705232W",
      "title": "Programming with Java"
    },
    {
      "author": "Kathy Sierra",
      "isbn": null,
      "firstPublishYear": 2003,
      "openLibraryKey": "/works/OL5756124W",
      "title": "Head first Java"
    },
    {
      "author": "DT Editorial Services R. Nageswara Rao",
      "isbn": null,
      "firstPublishYear": 2016,
      "openLibraryKey": "/works/OL24198547W",
      "title": "Core Java"
    },
    {
      "author": "Multatuli",
      "isbn": null,
      "firstPublishYear": 1868,
      "openLibraryKey": "/works/OL658280W",
      "title": "Max Havelaar of de koffijveilingen der Nederlansche handelmaatschappij"
    },
    {
      "author": "C. A. Backer",
      "isbn": null,
      "firstPublishYear": 1963,
      "openLibraryKey": "/works/OL1477988W",
      "title": "Flora of Java"
    },
    {
      "author": "Joshua Bloch",
      "isbn": null,
      "firstPublishYear": 2001,
      "openLibraryKey": "/works/OL6223299W",
      "title": "Effective Java"
    },
    {
      "author": "Thomas H. Cormen",
      "isbn": null,
      "firstPublishYear": 1990,
      "openLibraryKey": "/works/OL4781294W",
      "title": "Introduction to Algorithms"
```

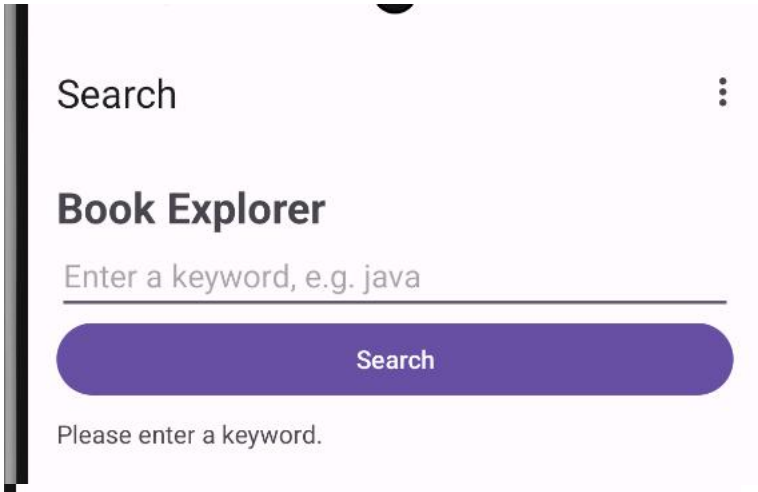

JSON response returned by the cloud web service.

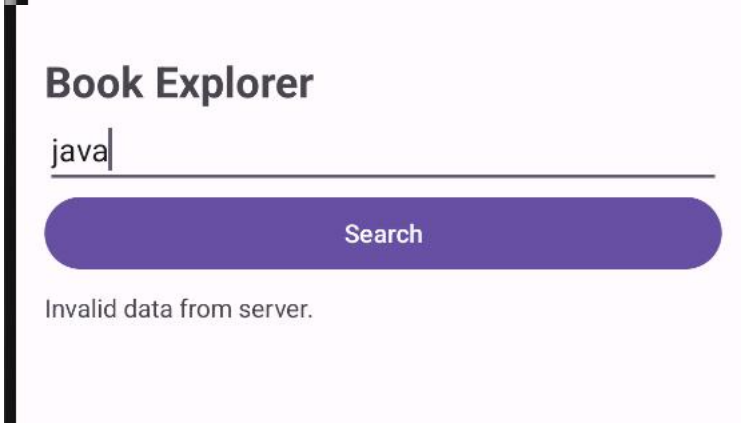
4. Requirement 3: Error Handling

My system handles several error conditions gracefully:

Empty user input in the Android app
Invalid or missing keyword on the server side
Mobile app network failure
Third-party API failure
Third-party API invalid or missing data

For example, the Android app shows a friendly message if the user submits an empty keyword, and the server returns an error JSON message if the keyword parameter is missing. The app also handles network failures without crashing. These behaviors address the project’s required error conditions.





5. Requirements 4 and 5: Logging and MongoDB Storage

My web service logs request information and stores it in MongoDB Atlas. The project requires logging useful information and storing it persistently in MongoDB.

For each mobile request, my application logs multiple useful pieces of information, including:

Timestamp of the request
Search keyword
User agent / phone information
Third-party API URL
Third-party API status code
Response latency
Number of results returned
Top result title
Server status

This satisfies the requirement to log at least six useful pieces of information covering the mobile request, the third-party API request/reply, and the reply to the mobile phone.

The logs are stored in MongoDB Atlas so they remain available across restarts of the application. This satisfies the database storage requirement.

6. Requirement 6: Dashboard

My web service includes a web-based operations dashboard accessible at:

This dashboard is designed for desktop/laptop browser use and displays both analytics and formatted logs, as required.

The dashboard displays at least three analytics, including:

Total Requests
Successful Requests
Average Latency (ms)
Latest Keyword

These analytics summarize how the service is being used and satisfy the requirement for at least three interesting operations analytics.

The dashboard also displays the full request logs in a formatted HTML table. The project instructions specifically require formatted logs and note that points are lost if logs are displayed directly as raw JSON or XML. My dashboard uses a table-based format instead.

https://ubiquitous-space-yodel-r97r65947gf5q6q-8080.app.github.dev/

Project 4 Dashboard

Showing the latest request logs from MongoDB.

Analytics

Metric	Value
Total Requests	6
Successful Requests	6
Average Latency (ms)	935
Latest Keyword	java

Request Logs

Timestamp	Keyword	Phone Model	API Status	Latency	Result Count	Top Result	Server Status
2026-04-10T21:47:05.377Z	java	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36	200	829 ms	10	Java	OK
2026-04-10T21:41:39.120Z	cat	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36	200	1393 ms	10	The Black Cat	OK
2026-04-10T20:37:00.014Z	python	Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_x86_64 Build/BE2A.250530.026.F3)	200	630 ms	10	Metamorphoses	OK
2026-04-10T20:35:52.014Z	java	Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_x86_64 Build/BE2A.250530.026.F3)	200	761 ms	10	Java	OK

7. Requirement 7: Deployment to GitHub Codespaces

My web service was packaged as a ROOT.war file and deployed to GitHub Codespaces. The project instructions specify Codespaces deployment using the provided container setup and a ROOT.war web application.

7.1 Deployment process

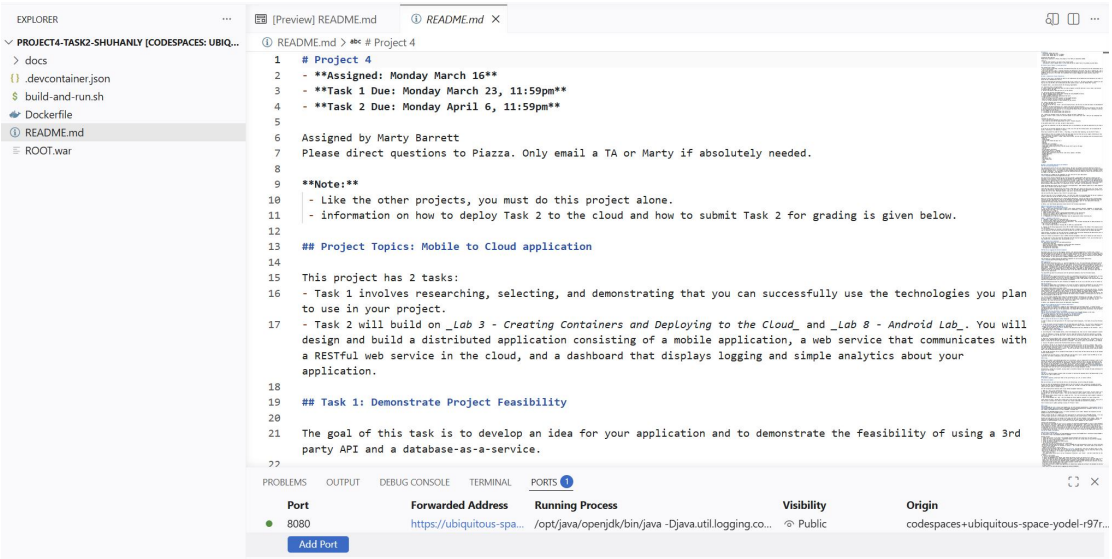
I packaged my web service into ROOT.war and placed it in the repository used for Codespaces deployment.

7.2 Public access

After the Codespace was created, I confirmed that port 8080 was available and changed the port visibility to Public so that the Android app could access the cloud service without GitHub authentication. This follows the project’s instructions for using Codespaces with Android.

7.3 Cloud endpoint

The Android app was updated to use the public Codespaces URL for the /searchBooks endpoint.



8. Submission Files

The following files are included in the repository for submission:

ROOT.war
IntelliJ IDEA web service project zip
Android Studio project zip
Writeup PDF

This matches the submission summary described in the project instructions.

9. Notes on Documentation

All source code files include author comments with my name and Andrew ID, following the project documentation standards. The project instructions state that missing author comments can result in a point deduction.